

Synaptiq

Production Architecture Document

HiDevs x Mastra Hackathon | Round 1 Submission

Mastra | Qdrant | Enkrypt AI | Next.js | PostgreSQL | Redis

AI-Powered Adaptive Tutoring Multi-Agent System

1. Problem & Impact

Indian K-12 and competitive-exam students lose 40-60% of new material within 48 hours because tutoring apps deliver static content, not personalized re-sequencing tied to forgetting curves and mastery gaps. Synaptiq is a TypeScript-native AI tutoring system that replaces one-size-fits-all chatbots with a Mastra-orchestrated multi-agent pipeline: it diagnoses learning style, sequences curriculum by Bloom level and IRT difficulty, retrieves grounded explanations from Qdrant, and gates every turn through Enkrypt AI before a student sees it. Target users are self-paced learners (JEE/NEET/CBSE). Measurable outcomes: mastery score delta, time-to-remediation, hallucination block rate.

2. Agent Workflows

A session flows through five stages:

- Onboarding - Identity Agent collects grade, goals, prior scores; writes profile to Postgres; seeds Qdrant.
- Intent routing - Supervisor Agent classifies turn (explain, practice, assess, review) and fans out.
- Adaptive planning - Style Detector, Forgetting-Curve Scheduler, IRT Calibrator run in parallel.
- Delivery - Tutor Agent generates explanations; calls Multimodal Tools on visual/kinesthetic hits.
- Assessment loop - Assessment Agent emits questions; wrong answers branch to Remediation Agent.

Agent Justifications

Agent	Pain Point Addressed
Supervisor Agent	Ambiguous questions waste 30%+ tokens without intent routing.
Learning Style Detector	Visual learners retain 65% more with diagrammed concepts.
Curriculum Sequencer	Exam syllabi are DAGs - topologically orders prerequisites.
Tutor Agent	Grounds every answer in Qdrant chunks, not parametric memory.
Assessment Agent	Forces active recall - the only signal IRT can calibrate on.
Remediation Agent	Walks prerequisite graph in Qdrant to find true failure point.
Progress Tracker	Surfaces mastery %, streak, next-review ETA from Postgres + Redis.
Bloom Classifier	Mis-tagged questions let students pass drills but fail exams.
IRT Calibrator	Adjusts difficulty theta to learner's zone of proximal development.
Forgetting-Curve Scheduler	60% retention loss in 48h (Ebbinghaus) - re-sequences before decay.
Spaced-Repetition Engine	Writes SM-2 intervals to Redis; surfaces due cards on session start.
RAG Retriever Agent	Hybrid dense+filtered Qdrant queries scoped to subject and grade.
Identity / Onboarding Agent	Bootstraps learner_profiles and seeds Qdrant namespaces per user.

3. Mastra Orchestration

All agents register on a single Mastra instance with shared PostgresStore (threads) and QdrantVector (semantic memory). Curriculum Workflow uses createWorkflow graph with .then(), .parallel(), and .branch(). Tutor Branch calls tutorAgent.generate() with memory threadId. Multimodal Tools are createTool definitions with requireToolApproval and workflow.suspend(). RAG uses createVectorQueryTool with QDRANT_PROMPT. Frontend subscribes to tutorAgent.stream() SSE.

Mastra Primitives Used

Primitive	Where Used
Mastra() + registerAgent	Root orchestrator instance
createWorkflow + .then/.branch/.parallel	Curriculum + assessment pipelines
createStep	Each workflow stage
Agent.generate() / .stream()	Tutor, Assessment, Supervisor agents
createTool	Diagram generator, code sandbox
createVectorQueryTool	RAG Retriever Agent
Memory + PostgresStore	Cross-session conversation continuity
QdrantVector (@mastra/qdrant)	Semantic memory backend
suspend() / resume()	Human-in-the-loop sandbox approval
Observability spans	enkrpt_check_id + qdrant_hit_ids attributes

4. Qdrant Memory & Retrieval

Collection	Contents	Retrieval
curriculum_chunks	Textbook chunks (1536-dim)	Filters: subject, grade, bloom_level
learner_misconceptions	Wrong-answer patterns per user	Filter: user_id; upserted on fail
session_summaries	Compressed turn summaries	Vector + thread_id filter
multimodal_assets	Diagram caption embeddings	Linked to chunk_id for re-use

Ingestion: PDFs -> MDocument chunker -> embed() -> store.upsert(). Redis (Upstash) caches hot results (TTL 15 min) and SM-2 review queues. After remediation, misconception vectors upserted for faster future retrieval.

5. Enkrpt AI Evaluation Layer

Enkrpt is a first-class pipeline stage. Every turn passes through policy bundle synaptiq-tutor-v1.

Pipeline Stages

```

Student Input
-> STAGE 1 Pre-Ingress: PII mutate + detect (jailbreak/toxicity)
-> Mastra Supervisor -> Agents -> Qdrant RAG
-> STAGE 2 Post-Retrieval: adherence check (response vs context)
-> Tutor Agent .generate() / .stream()
-> STAGE 3 Post-Generation: hallucination + relevancy + PII mutate
-> STAGE 4 Policy Gate: /guardrails/policy/detect
-> Stream to Frontend
  
```

Failure Actions

Check	Threshold	Action
PII (request)	entity detected	Mutate - store enkrypt_key in Redis
Jailbreak/injection	score > 0.85	Block - log to safety_events
Toxicity/NSFW	score > 0.8	Block - escalate for minors
Adherence	score < 0.7	Retry widen topK; then escalate
Hallucination	is_hallucination: 1	Retry strict_grounding; 2nd fail -> suspend()
Relevancy	score < 0.6	Retry with Supervisor re-routing
PII (response)	entity detected	Mutate before streaming

6. Frontend

Next.js 15 (App Router) on Vercel. Pages: /onboarding, /learn/[topicId], /assess/[sessionId], /progress. Split-pane learn view: SSE chat + Mermaid/Monaco panel. TanStack Query for API state. Clerk/NextAuth JWT auth. Safety Badge (green/amber/red) from Enkrypt stage-4. Progress dashboard shows IRT theta, Bloom breakdown, due review cards.

7. Backend & APIs

Method	Path	Purpose
POST	/api/v1/sessions	Create session; Onboarding Workflow
POST	/api/v1/sessions/:id/message	Full Enkrypt + Maestra pipeline; SSE
GET	/api/v1/sessions/:id/stream	Resume in-flight Tutor stream
POST	/api/v1/assessments/:id/submit	Assessment Workflow; IRT update
GET	/api/v1/curriculum/:userId	Sequenced topic DAG
GET	/api/v1/progress/:userId	Mastery, streak, SM-2 due cards
POST	/api/v1/reviews/:cardId/complete	Mark spaced-repetition card done
POST	/api/v1/ingest	Admin: PDF -> Qdrant upsert
GET	/api/v1/safety/events/:userId	Enkrypt audit log
POST	/api/v1/workflows/:id/resume	HITL resume after suspend()

8. Database

PostgreSQL (Neon)

- users - id, email, grade, exam_target, learning_style
- learner_profiles - user_id, irt_theta, bloom_distribution, style_scores
- sessions - id, user_id, thread_id, status, started_at
- messages - id, session_id, role, content_redacted, enkrypt_key, safety_status
- assessments - id, session_id, topic_id, bloom_level, difficulty_theta, correct
- mastery_records - user_id, topic_id, mastery_pct, last_practiced
- spaced_repetition_cards - id, user_id, topic_id, interval_days, ease_factor, due_at
- safety_events - id, session_id, stage, check_type, score, action

- curriculum_topics - id, subject, title, prerequisite_ids, bloom_level

Redis (Upstash)

- session:{id}:encrypt_key - PII deanonymization
- reviews:due:{userId} - SM-2 sorted set
- Qdrant query cache (TTL 15 min); rate limits 100 req/min/user

9. Deployment

Layer	Host	Rationale
Frontend	Vercel	Edge SSR, PR preview deploys
Mastra API	Railway (Docker)	Long-running workers for .stream()
PostgreSQL	Neon	Branching DB per env
Redis	Upstash	Serverless, same region as Railway
Qdrant	Qdrant Cloud	Separate collections per env
Enkrypt AI	api.enkryptai.com	API key per env in secrets

CI/CD

git push -> GitHub Actions (lint, typecheck, test) -> Docker build -> Railway deploy. Vercel auto-deploys frontend. main -> staging; tag v* -> production. Mastra agents scale horizontally on Railway replicas; PostgresStore ensures suspend/resume across replicas.

10. Judging Alignment

Criterion	Weight	Synaptiq Coverage
Mastra Integration	25%	Workflows, Agents, Tools, Memory, VectorQueryTool, suspend/resume
Qdrant Integration	20%	4 collections, metadata filters, misconception write-back
Enkrypt AI Coverage	20%	4-stage pipeline, 7 policies, block/retry/escalate matrix
Agent Output Quality	20%	RAG-grounded Tutor + IRT Assessment + remediation loop
Problem Impact	15%	Forgetting-curve scheduling, IRT, 48-hour retention targeting